1.5 Rekursive Algorithmen

Ruft sich eine Methode selbst mit anderem Parameter auf, ist sie **rekursiv**. *Website: Station 1.5, Labor und Projekt*

Aufgabe 1 *Hin und zurück*

```
void countdown(int n) {
   if (n == 0) {
      println("Start!");
   } else {
      println("hin " + n);
      countdown(n - 1);
      println("zurück " + n);
   }
}
```

a) Sage vorher, was `countdown(3)` ausgibt:

hin 3, hin 2, hin 1, Start!,

zurück 1, zurück 2, zurück 3

b) Wenn „Start!“ erscheint, warten 3 Aufrufe. Als Erstes gibt countdown(1) „zurück“ aus.

Rekursive Algorithmen

Drei Fragen: ① **Abbruchbedingung**: Wann steht die Antwort sofort fest? ② **Verkleinern**: Mit welchem kleineren Problem ruft sich die Methode auf? ③ **Zusammensetzen**: Was macht sie mit der Antwort?

Wartende Aufrufe liegen auf dem Aufrufstapel, der zuletzt aufgerufene wird als Erster fertig.

Linear: höchstens ein rekursiver Aufruf je Durchlauf, Darstellung: Aufruftabelle. **Verzweigt:** mehrere rekursive Aufrufe je Durchlauf, Darstellung: Aufrufbaum.

Aufgabe 2 *Wenn die Rekursion nicht endet (Projekt, Klasse Experiment)*

a) Starte jede Fassung mit `n = 3` und notiere die ersten fünf Werte von `n`. **b)** Welche der drei Fragen ①–③ ist verletzt? Begründe kurz.

	Kern des Codes	a) erste Werte von `n`	b) verletzt, weil …
A	`(kein if) fassungA(n - 1)`	3, 2, 1, 0, -1	①: Es gibt keine Abbruchbedingung.
B	`if (n == 0) … fassungB(n - 2)`	3, 1, -1, -3, -5	②: Die 0 wird übersprungen.
C	`if (n == 0) … fassungC(n)`	3, 3, 3, 3, 3	②: n wird nicht kleiner.

c) TODO 2: Fassung B endet für jede Startzahl mit `if (` n <= 0 `)`. Jeder Aufruf muss der Abbruchbedingung näher kommen, sonst bricht Java mit einem StackOverflowError ab.

Aufgabe 3 *Aufruftabelle zum ggT*

```
int ggT(int a, int b) {
   if (b == 0) {
      return a;
   }
   return ggT(b, a % b);
}
```

a) Fülle die Tabelle für `ggT(84, 36)` aus. Rückgaben von unten nach oben!

Aufruf	a	b	a % b	Abbruch?	Rückgabe
`ggT(84, 36)`	84	36	12	nein	12
`ggT(36, 12)`	36	12	0	nein	12
`ggT(12, 0)`	12	0	–	ja	12

b) `ggT(36, 84)` ruft als Erstes ggT(84, 36) auf: Die Parameter werden vertauscht. Insgesamt sind es 4 Aufrufe.

Aufgabe 4 *Lineare Rekursion planen und programmieren (TODO 4a–c)*

a) Plane jede Methode mit den drei Fragen, bevor du sie im Projekt programmierst. Schreibe Java-Ausdrücke.

Methode	① Abbruch: Bedingung → Antwort	② rekursiver Aufruf	③ zusammensetzen
`quersumme(n)`	`n < 10` → `n`	`quersumme(n / 10)`	`n % 10` + Antwort
`umrechnen(n, basis)`	`n < basis` → Ziffer `n`	`umrechnen(n / basis, basis)`	Antwort + Ziffer für `n % basis`
`umdrehen(s)`	`s.length() <= 1` → `s`	`umdrehen(s.substring(1))`	Antwort + `s.charAt(0)`

b) Trage die Werte von `n` in allen Aufrufen und das Ergebnis ein (Labor oder Projekt):

`umrechnen(22, 2)`: n = 22, 11, 5, 2, 1 Ergebnis ”10110”

`umrechnen(22, 3)`: n = 22, 7, 2 Ergebnis ”211”

Je größer `basis`, desto weniger Aufrufe und desto kürzer das Ergebnis.

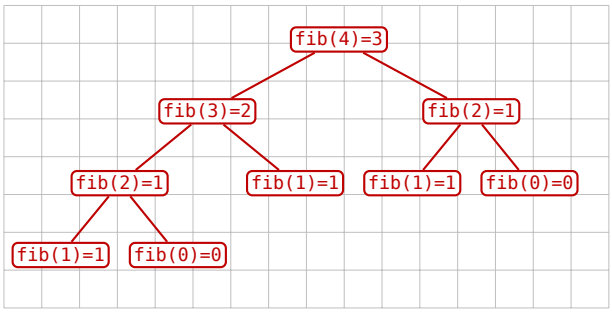
Aufgabe 5 *Fibonacci: verzweigte Rekursion (TODO 5a, 5b)*

Definition: $fib(0) = 0,\ fib(1) = 1,\ fib(n) = fib(n-1) + fib(n-2)$ für $n \geq 2$. Folge: 0, 1, 1, 2, 3, 5, 8, …

a) Ergänze den Code.

```
int fib(int n) {
   aufrufe++;                    // TODO 5b
   if (n == 0 ) { return 0; }
   if (n == 1) { return 1 ; }
   return fib(n - 1) + fib(n - 2) ;
}
```

b) Zeichne den Aufrufbaum von `fib(4)` und schreibe jede Rückgabe an ihren Aufruf.



Die beiden `if` sind die Abbruchbedingungen.

`fib` ist verzweigt rekursiv, weil jeder Aufruf (außer 0, 1) zwei weitere auslöst.

c) Zähle mit dem Projekt (Abschnitt Aufgabe 5):

n	5	10	15	20	25
`fib(n)`	5	55	610	6765	75025
Aufrufe	15	177	1973	21891	242785

Wird `n` um 5 größer, steigt die Zahl der Aufrufe etwa auf das 11 -Fache.

d) In `fib(5)` wird `fib(2)` 3 -mal berechnet.

e) Was passiert bei `fib(-1)`? Begründe.

n wird -1, -2, -3, … und trifft nie 0 oder 1: Die Rekursion endet nicht, StackOverflowError.

Aufgabe 6 *Der Tresor: vier Algorithmen in Reihe (TODO 6)*

Notiere die Zwischenwerte. `fib(17)` = 1597 → Quersumme 22 → im Zweiersystem ”10110” → rückwärts gelesen (Code) ”01101”

Die Nachricht im Tresor lautet: TRESOR OFFEN! UM REKURSION ZU VERSTEHEN, MUSST DU ZUERST REKURSION VERSTEHEN.